Tugas Besar 1 IF2211 Strategi Algoritma

Pemanfaatan Algoritma Greedy dalam pembuatan bot permainan Battlecode 2025

Disusun Oleh:

Narendra Dharma Wistara Marpaung	(13524044)
Nashiruddin Akram	(13524090)
Manuel Thimoty Silalahi	(13524102)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Daftar Isi

1 Deskripsi Tugas **1**

2 Landasan Teori **1**
2.1 Dasar Teori Algoritma Greedy 1
2.2 Cara Kerja Program Bot 2
2.3 Menjalankan Program Bot 3

3 Aplikasi Strategi Greedy **3**
3.1 Mapping Persoalan Battlecode ke Elemen Algoritma Greedy 3
3.2 Eksplorasi 3 Alternatif Solusi Greedy 5
3.2.1 Alternatif Solusi 1: Greedy Territorial Expansion 5
3.2.2 Solusi Alternatif 2 (Pembagian Role) 6
3.2.3 Alternatif 3 (Greedy Prioritas Aksi): 7
3.3 Analisis Efisiensi dan Efektivitas 8
3.4 Strategi Greedy Terpilih 8

4 Implementasi dan Pengujian **9**
4.1 Implementasi 3 Alternatif Solusi 9
4.2 Struktur Data, Fungsi, dan Prosedur 15
4.3 Pengujian (Bot Utama vs Bot Alternatif) 17
4.4 Analisis Hasil Pengujian 19

5 Kesimpulan dan Saran **20**
5.1 Kesimpulan 20
5.2 Saran 20

Lampiran **21**

Daftar Pustaka **22**

1 Deskripsi Tugas

Tugas besar ini berfokus pada perancangan bot Battlecode 2025 berbasis algoritma greedy. Battlecode adalah permainan strategi waktu nyata di mana Tim akan menulis kode untuk bot/tower. Pemain harus mengelola pasukan bot secara strategis dan mengendalikan cara bot-bot bekerja sama untuk mengalahkan tim lawan. Kecerdasan buatan, penelusuran jalur, algoritma terdistribusi, dan komunikasi akan sangat penting disini untuk membuat pasukan sesaing mungkin. Namun, Anda hanya memiliki sumber daya komputasi terbatas per giliran, jadi Anda harus merencanakan strategi, mengoptimalkan, dan berkompromi sesuai kebutuhan. Setiap tim diminta memodelkan persoalan permainan sebagai persoalan optimasi keputusan bertahap, lalu mengimplementasikan strategi yang dapat mengambil aksi terbaik secara lokal pada setiap giliran.

Secara umum, deliverable tugas terdiri dari:

- Implementasi bot utama berbasis greedy.
- Eksplorasi minimal tiga alternatif strategi greedy dengan heuristik berbeda.
- Pengujian bot utama melawan bot alternatif dalam format 1 vs 1 minimal tiga kali.
- Analisis hasil eksperimen, termasuk kondisi saat strategi berhasil maupun gagal mencapai hasil optimal.

Fokus evaluasi laporan mencakup ketepatan pemetaan elemen greedy, kualitas perancangan heuristik, kualitas implementasi, dan kualitas analisis hasil pengujian.

2 Landasan Teori

2.1 Dasar Teori Algoritma Greedy

Berdasarkan literatur oleh Rinaldi Munir, algoritma *greedy* merupakan metode yang digunakan untuk memecahkan masalah optimasi (*optimization problems*). Prinsip utama algoritma ini adalah mengambil keputusan yang terbaik pada setiap langkah tanpa mempedulikan konsekuensi di masa depan (*take what you can get now!*), dengan harapan bahwa rangkaian pilihan lokal tersebut akan mengarah pada solusi optimum global.

Menurut Rinaldi Munir, algoritma *greedy* disusun oleh lima elemen utama sebagai berikut:

- **Himpunan Kandidat (*Candidate Set*)**: Berisi elemen-elemen pembentuk solusi yang akan dipilih pada setiap langkah.
- **Himpunan Solusi (*Solution Set*)**: Berisi kandidat-kandidat yang telah terpilih sebagai bagian dari solusi masalah.
- **Fungsi Seleksi (*Selection Function*)**: Fungsi yang memilih kandidat terbaik berdasarkan kriteria *greedy* tertentu (pilihan *locally optimal*).

- **Fungsi Kelayakan (*Feasible Function*)**: Fungsi untuk memeriksa apakah kandidat yang dipilih tidak melanggar batasan (*constraints*) yang ada.
- **Fungsi Objektif (*Objective Function*)**: Fungsi yang hendak dimaksimalisasi atau diminimalisasi nilainya.

Dua syarat utama agar algoritma *greedy* dapat menghasilkan solusi yang benar-benar optimal secara global adalah:

1. *Greedy choice property*: Sebuah solusi optimal global dapat dicapai dengan memilih solusi optimum lokal pada setiap langkahnya.
2. *Optimal substructure*: Masalah tersebut memiliki struktur dimana solusi optimal dari masalah besar mengandung solusi optimal dari sub-masalahnya.

Kelebihan utama algoritma ini terletak pada efisiensi waktu eksekusi dan kesederhanaan implementasinya, yang sangat krusial untuk sistem *real-time* seperti *bot game*. Namun, keterbatasannya adalah algoritma ini tidak selalu menjamin tercapainya solusi optimum global, terutama pada konfigurasi permainan atau peta yang memiliki kompleksitas tinggi.

2.2 Cara Kerja Program Bot

Pada permainan Battlecode, setiap robot menjalankan program secara independen pada setiap ronde permainan. Setiap ronde, robot akan membaca kondisi lingkungan di sekitarnya melalui mekanisme *sensing*, kemudian menentukan aksi terbaik yang dapat dilakukan pada giliran tersebut. Aksi yang dapat dilakukan oleh robot antara lain bergerak, menyerang atau mewarnai petak, membangun struktur, mentransfer sumber daya, serta berkomunikasi dengan robot lain dalam tim.

Secara umum, alur kerja bot yang diimplementasikan dalam program ini terdiri dari beberapa tahap utama. Pertama, robot membaca *state* permainan yang tersedia melalui API Battlecode, seperti posisi robot sendiri, posisi robot musuh yang terdeteksi, informasi peta di sekitar, serta kondisi sumber daya yang dimiliki. Informasi ini menjadi dasar bagi robot untuk mengambil keputusan pada giliran tersebut.

Setelah informasi lingkungan diperoleh, bot akan membangkitkan sejumlah aksi kandidat yang mungkin dilakukan oleh robot. Aksi kandidat ini dapat berupa pergerakan ke beberapa arah yang memungkinkan, menyerang petak tertentu, atau melakukan aksi lain yang valid sesuai dengan aturan permainan. Setiap aksi kandidat kemudian dievaluasi menggunakan fungsi heuristik yang dirancang berdasarkan prinsip algoritma *greedy*.

Dalam implementasi algoritma *greedy*, bot akan memilih aksi yang memberikan nilai keuntungan lokal terbaik pada saat itu. Nilai tersebut dihitung menggunakan fungsi evaluasi yang mempertimbangkan berbagai faktor, seperti potensi memperluas wilayah yang diwarnai, peluang menyerang atau mengganggu musuh, serta efisiensi penggunaan sumber daya. Dari seluruh aksi kandidat yang tersedia, bot memilih aksi dengan nilai evaluasi tertinggi yang tetap memenuhi batasan atau aturan permainan.

Setelah aksi terbaik dipilih, robot akan mengeksekusi aksi tersebut menggunakan fungsi yang disediakan oleh API Battlecode. Setelah seluruh proses selesai dilakukan, robot akan mengakhiri gilirannya untuk menunggu ronde berikutnya. Proses ini kemudian akan diulang kembali pada ronde selanjutnya selama robot masih berada di dalam permainan.

2.3 Menjalankan Program Bot

Untuk menjalankan program bot, proyek Battlecode terlebih dahulu harus dikompilasi menggunakan sistem build yang telah disediakan. Pada sistem operasi Windows, proses kompilasi dapat dilakukan dengan menjalankan perintah berikut pada terminal:

- Kompilasi proyek: `.\gradlew.bat build`

Setelah proses kompilasi berhasil, pertandingan dapat dijalankan menggunakan perintah:

- Menjalankan client: `.\client\Stima Battle Client`

Perintah tersebut akan membuka aplikasi *Battlecode Client* yang menyediakan antarmuka grafis untuk menjalankan dan memantau pertandingan. Setelah aplikasi terbuka, pengguna dapat memilih tab *Runner* yang terletak pada bagian kiri atas antarmuka.

Pada tab *Runner*, pengguna dapat menentukan konfigurasi pertandingan yang akan dijalankan. Pengguna dapat memilih bot yang akan bertanding melalui pilihan *Team A* dan *Team B*. Daftar bot yang tersedia pada pilihan tersebut bergantung pada bot yang telah dibuat dan terdaftar di dalam proyek.

Selain itu, pada bagian yang sama juga tersedia daftar peta (*maps*) yang dapat digunakan untuk menjalankan pertandingan. Pengguna dapat memilih satu atau lebih peta dari daftar tersebut sesuai dengan kebutuhan pengujian.

Setelah konfigurasi tim dan peta ditentukan, pertandingan dapat dimulai dengan menekan tombol *Run*. Engine permainan kemudian akan menjalankan simulasi pertandingan dan menampilkan jalannya permainan secara visual.

Untuk melihat informasi statistik pertandingan secara lebih rinci, pengguna dapat berpindah ke tab *Game*. Pada tab ini ditampilkan berbagai informasi penting selama pertandingan berlangsung, seperti persentase wilayah yang berhasil diwarnai (*coverage area*), jumlah uang (*chips*), jumlah robot yang aktif, jumlah menara yang masih bertahan, serta statistik lain yang berkaitan dengan kondisi permainan.

3 Aplikasi Strategi Greedy

3.1 Mapping Persoalan Battlecode ke Elemen Algoritma Greedy

Untuk menerapkan algoritma *greedy* pada permainan Battlecode 2025, permasalahan permainan terlebih dahulu dipetakan ke dalam elemen-elemen utama algoritma

greedy. Tujuan utama permainan ini adalah memenangkan pertandingan dengan cara mewarnai wilayah peta sebanyak mungkin atau mengalahkan seluruh unit lawan. Oleh karena itu, strategi yang dirancang harus berfokus pada pengambilan keputusan lokal terbaik pada setiap giliran robot untuk memaksimalkan penguasaan wilayah dan efektivitas unit.

Berdasarkan kerangka algoritma greedy, elemen-elemen yang digunakan dalam pemodelan persoalan Battlecode adalah sebagai berikut.

1. Himpunan Kandidat (*Candidate Set*) Himpunan kandidat merupakan kumpulan aksi yang mungkin dilakukan oleh sebuah robot pada suatu giliran. Dalam konteks Battlecode, aksi kandidat yang dapat dipertimbangkan antara lain:

- Bergerak ke salah satu petak tetangga yang dapat dilalui.
- Menyerang atau mewarnai petak tertentu dalam jangkauan aksi.
- Menghapus warna lawan pada petak tertentu.
- Membangun atau menyelesaikan pola struktur tertentu (misalnya menara atau pola sumber daya).
- Melakukan aksi pendukung seperti transfer sumber daya antar unit.

Semua aksi yang secara aturan permainan memungkinkan untuk dilakukan oleh robot pada ronde tersebut akan dimasukkan ke dalam himpunan kandidat.

2. Himpunan Solusi (*Solution Set*) Himpunan solusi merupakan kumpulan aksi yang telah dipilih oleh algoritma sebagai keputusan yang akan dieksekusi oleh robot. Dalam permainan ini, himpunan solusi direpresentasikan oleh urutan aksi yang diambil oleh robot pada setiap ronde permainan.

Karena permainan berlangsung secara bertahap dan setiap robot mengambil keputusan secara independen pada setiap ronde, solusi dibangun secara iteratif melalui pemilihan aksi terbaik pada setiap giliran.

3. Fungsi Seleksi (*Selection Function*) Fungsi seleksi digunakan untuk memilih aksi terbaik dari himpunan kandidat berdasarkan kriteria tertentu. Dalam implementasi greedy pada bot Battlecode, fungsi seleksi menggunakan nilai heuristik untuk menilai setiap aksi kandidat.

Contoh kriteria yang dapat digunakan dalam fungsi seleksi antara lain:

- Seberapa besar potensi aksi tersebut dalam menambah jumlah wilayah yang diwarnai.
- Kedekatan dengan wilayah yang belum dikuasai.
- Potensi mengganggu atau merusak wilayah yang telah dikuasai oleh lawan.
- Efisiensi penggunaan sumber daya seperti *paint* dan *chips*.

Aksi dengan nilai heuristik tertinggi akan dipilih sebagai keputusan pada ronde tersebut.

4. Fungsi Kelayakan (*Feasible Function*) Fungsi kelayakan digunakan untuk memastikan bahwa aksi kandidat yang dipilih tidak melanggar aturan permainan. Dalam konteks Battlecode, suatu aksi dianggap layak apabila memenuhi kondisi berikut:

- Robot memiliki *cooldown* aksi atau pergerakan yang memungkinkan untuk melakukan aksi tersebut.
- Petak tujuan pergerakan tidak ditempati oleh unit lain dan dapat dilalui.
- Target aksi berada dalam jangkauan kemampuan robot.
- Robot memiliki sumber daya yang cukup untuk melakukan aksi tersebut.

Aksi yang tidak memenuhi kriteria kelayakan akan dieliminasi dari himpunan kandidat sebelum proses seleksi dilakukan.

5. Fungsi Objektif (*Objective Function*) Fungsi objektif mendefinisikan tujuan utama yang ingin dicapai oleh algoritma. Dalam permainan Battlecode, tujuan utama adalah memaksimalkan penguasaan wilayah peta melalui pewarnaan petak.

Dengan demikian, fungsi objektif dapat dirumuskan sebagai upaya untuk memaksimalkan jumlah petak yang berhasil dikuasai oleh tim. Selain itu, faktor tambahan seperti mempertahankan unit, membangun menara, serta mengganggu perkembangan lawan juga dapat berkontribusi secara tidak langsung terhadap pencapaian tujuan utama tersebut.

Melalui pemetaan persoalan ini, algoritma greedy dapat digunakan untuk memilih aksi terbaik pada setiap langkah secara lokal dengan harapan bahwa rangkaian keputusan tersebut akan menghasilkan performa permainan yang optimal secara keseluruhan.

3.2 Eksplorasi 3 Alternatif Solusi Greedy

3.2.1 Alternatif Solusi 1: Greedy Territorial Expansion

Strategi ini berfokus pada ekspansi wilayah secepat mungkin dengan memaksimalkan jumlah petak yang diwarnai pada setiap ronde. Bot tidak secara aktif membangun tower baru karena proses pembangunan dianggap kurang efektif dibandingkan langsung memperluas wilayah. Mengingat jumlah ronde permainan yang terbatas, pembangunan tower dianggap tidak efektif karena setelah dibangun, belum tentu dapat dinikmati hasilnya (Chips, Paint, dan Sumber Daya lainnya).

Bot memprioritaskan pergerakan menuju area dengan konsentrasi petak kosong yang tinggi agar ekspansi wilayah lebih efisien. Selain itu, untuk mengantisipasi lawan yang membangun Tower, bot akan memprioritaskan penghancuran menara musuh segera setelah Tower tersebut terdeteksi atau selesai dibangun. Dengan pendekatan ini, setiap keputusan robot selalu memilih aksi yang memberikan keuntungan lokal terbesar dalam memperluas wilayah tim (Greedy).

3.2.2 Solusi Alternatif 2 (Pembagian Role)

Setelah dilakukan eksplorasi terhadap permainan Battlecode, salah satu kelemahan yang dianalisis adalah terlalu sedikitnya *role* atau jenis bot yang terdapat pada permainan. Sehingga, solusi yang ditawarkan merupakan pembagian jenis robot *soldier* ke beberapa peran lainnya. Selain itu, strategi ini juga membagi permainan ke dalam dua fase. Fase pertama merupakan fase eksplorasi dan pembangunan *tower*. Fase ini memanfaatkan informasi lebar dan panjang *map*, serta fakta bahwa *map* bersifat simetris, entah secara horizontal, vertikal, atau keduanya. Fase pertama ini dilaksanakan dari awal permainan hingga ronde ke-150. Pada fase pertama ini, *soldier* dibagi menjadi dua peran, yaitu *Searcher* dan *Builder*. *Searcher* bertugas untuk menemukan *ruins*, lalu mencatat keberadaan *ruins* tersebut. Setelah lokasi *ruins* dicatat, lokasi tersebut disampaikan ke *tower* terdekat untuk ditugaskan ke seorang *Builder*. *Builder* bertugas untuk membangun sebuah *tower* di lokasi *ruins* yang sudah diberikan oleh *Searcher*. Pembangunan *tower* juga hanya bergantian antara *Paint Tower* dan *Money Tower*, tanpa membangun *Defense Tower* Pada fase pertama, untuk efisiensi *paint* dan *chips*, hanya di-*spawn* dengan urutan: *Searcher, Searcher, Builder, Builder, Searcher*. Pada fase pertama, tidak ada bot yang di-*spawn* selain lima bot dengan urutan tersebut.

Fase pertama selesai pada ronde ke-150. Setelah itu, algoritma masuk ke fase kedua. Setelah fase kedua, *tower* tidak lagi mencoba *spawn* berdasarkan urutan tersebut, tetapi dengan urutan *Soldier, Splasher, Mopper* sampai permainan selesai. Namun, setiap ronde selalu dicek apakah terdapat *ruins* yang ditemukan dan belum dibangun. Jika terdapat *ruins* yang belum dibangun, maka *Builder* kembali dikerahkan. Selain itu, fase dua ini membagi *soldier* kembali ke dua peran, yaitu *Scout* dan *Connector*. Keduanya di-*spawn* secara bergantian. Tugas *Scout* adalah untuk mencari wilayah kita yang diwarnai musuh untuk diambil alih oleh *Mopper* atau *Splasher*. Sedangkan, *connector* adalah untuk menciptakan jalur cat dari *tower*. Ini menjadi garis bantu yang diikuti oleh bot lain, karena dalam pewarnaan petak digunakan nilai tambah, jika petak yang diwarnai berada di sebelah petak sendiri. Di masing-masing *role* yang ditugaskan, digunakan algoritma *greedy* untuk mencapai tujuan akhir, kecuali untuk role *Searcher* dan *Scout* yang memanfaatkan kesimetrisan *map* dalam algoritmanya. Secara singkat, algoritma yang digunakan dari masing-masing *role* adalah sebagai berikut.

- *Searcher*: Melakukan eksplorasi dengan memanfaatkan kesimetrisan *map*, lalu membagi peta menjadi beberapa sektor untuk di-*explore* secara *zig-zag*.
- *Builder*: Mencari *ruins* yang memiliki jarak terpendek. Lalu, fokus ke *ruins* tersebut hingga pola 5×5 terbentuk dan *tower* dibangun.
- *Scout*: Berkeliling peta berdasarkan sudut kuadran, pusat, dan tepi *map*. Robot ini memprioritaskan pergerakan ke lokasi yang baru saja dibersihkan oleh Mopper untuk segera diwarnai kembali dan mengganggu tower musuh jika ditemukan.
- *Connector*: Menggunakan fungsi heuristik berdasarkan, bersebelahan (*adjacent*) ke cat kawan, menimpa cat musuh, apakah petak tersebut sudah diwarnai atau belum.

- *Splasher*: Menggunakan fungsi heuristik berdasarkan, petak apa yang dicat (punya musuh, punya sendiri, atau petak kosong). Serta diberikan juga bonus jika ledakan mengenai *tower* lawan. Ada minimal skor agar *Splasher* melakukan serangan dan minimal petak yang dicetak agar *Splasher* tetap efektif.
- *Mopper*: Diprioritaskan apabila terdapat musuh di petak tersebut untuk diambil catnya. Menggunakan fungsi heuristik berdasarkan jumlah petak yang memiliki cat musuh.

3.2.3 Alternatif 3 (Greedy Prioritas Aksi):

Salah satu keuntungan dari implementasi pada algoritma ini adalah penggunaan bytecode yang sedikit. Pada unit Mopper, strategi ini terlihat sangat eksplisit pada fase eksekusi aksi utamanya. Program akan mengevaluasi ketersediaan serangan mulai dari potensi tingkat kerusakan tertinggi hingga utilitas dukungan logistik terendah. Mopper pertama-tama mengecek validitas eksekusi Mop Swing untuk memberikan damage area secara masif. Jika cooldown atau jarak tidak memenuhi syarat, algoritma akan turun ke pengecekan serangan Mop biasa untuk mencuri cat musuh. Pada prioritas terbawah, jika tidak ada musuh sama sekali, ia baru akan mentransfer cat ke unit sekutu.

Algoritma *greedy* pada unit Soldier diterapkan sebagai berikut. Pada early game atau disini saya set sebelum jumlah round menyentuh x, Soldier secara greedy akan selalu memprioritaskan penanganan reruntuhan atau ruin menjadi menara pertahanan di atas segala instruksi lainnya. Begitu sensor mendeteksi keberadaan ruin yang belum diduduki dan suplai cat pada inventory robot tidak kosong, Soldier akan mengabaikan target eksplorasi sebelumnya, berjalan menempel ke reruntuhan tersebut, dan menggunakan sisa action cooldown-nya untuk mengecat pola menara. Baru setelah ronde melewati batas x sehingga flag transisi fase aktif, atau ketika area sensor benar-benar bersih dari reruntuhan, algoritma akan berubah menjadi ekspansi penyebaran guna memperluas penguaasaan wilayah peta secara masif. Dasar dari ide algoritma ini adalah bahwa di *early game*, membentuk tower dari ruin tergolong hal yang krusial tanpa distraksi musuh. *Note : pada implementasi ini,* $x = 500$.

Splasher secara *greedy* memecah urutan prioritasnya menjadi dua blok independen yang dievaluasi secara terpisah pada tiap satuan waktu. Pada blok serangannya, algoritma menyeleksi kandidat koordinat secara greedy dengan menghitung bobot skor area; ia akan mengabaikan petak kosong jika ada koordinat dengan kalkulasi tumpukan skor musuh yang lebih besar untuk disiram cat. Sementara itu pada blok pergerakannya, eksekusi diprioritaskan berdasarkan ambang batas suplai cat internal. Jika sisa cat berada di bawah 20 unit, Splasher seketika memotong fungsi pertempurannya dan memprioritaskan arah gerak untuk pulang ke menara sekutu terdekat guna melakukan pengisian ulang. Apabila syarat refill tersebut tidak terpenuhi (cat masih berlimpah), baru algoritma memprioritaskan pergerakan untuk mengejar musuh terdekat yang tertangkap radar, disusul prioritas mencari ubin kosong untuk diwarnai, dan jatuh ke fallback penjelajahan kompas apabila tidak ada entitas relevan yang ditemukan.

3.3 Analisis Efisiensi dan Efektivitas

Analisis efisiensi dan efektivitas dari kumpulan alternatif solusi *greedy* yang dirumuskan dilakukan dengan mempertimbangkan kompleksitas pengambilan keputusan, penggunaan sumber daya, serta dampaknya terhadap penguasaan wilayah pada permainan.

- **Alternatif 1 – Greedy Territorial Expansion**
  Strategi ini memiliki kompleksitas keputusan yang rendah dan penggunaan *bytecode* yang efisien karena robot hanya memprioritaskan ekspansi wilayah. Namun, strategi ini kurang efektif dalam jangka panjang karena tidak memanfaatkan pembangunan *tower* sebagai sumber daya tambahan.

- **Alternatif 2 – Pembagian Role**
  Strategi ini lebih efektif dalam koordinasi tim karena setiap robot memiliki peran yang spesifik dan permainan dibagi ke dalam dua fase. Pendekatan ini meningkatkan efisiensi eksplorasi, pembangunan *tower*, serta kontrol wilayah. Akan tetapi, strategi ini memiliki kompleksitas implementasi yang lebih tinggi.

- **Alternatif 3 – Greedy Prioritas Aksi**
  Strategi ini memprioritaskan efisiensi eksekusi dengan menentukan urutan aksi yang jelas bagi setiap unit. Pendekatan ini relatif hemat *bytecode* dan stabil dalam eksekusi, tetapi kurang adaptif terhadap kondisi permainan yang berubah secara signifikan.

Secara umum, ketiga alternatif memiliki karakteristik yang berbeda. Alternatif pertama unggul dalam kesederhanaan, alternatif kedua dalam koordinasi strategi tim, dan alternatif ketiga dalam efisiensi eksekusi algoritma.

3.4 Strategi Greedy Terpilih

Berdasarkan analisis dan hasil pengujian terhadap ketiga alternatif solusi yang telah dirumuskan, strategi yang dipilih untuk diimplementasikan adalah **Alternatif 2 (Pembagian Role)**. Pemilihan strategi ini utamanya didasarkan pada hasil eksperimen yang menunjukkan bahwa strategi tersebut memiliki *win rate* yang lebih tinggi dibandingkan dua strategi alternatif lainnya ketika diuji secara langsung.

Strategi ini juga mampu memberikan koordinasi tim yang lebih baik melalui pembagian *role* pada setiap unit, seperti *Searcher*, *Builder*, *Scout*, dan *Connector*. Dengan pembagian tugas yang jelas, proses eksplorasi, pembangunan *tower*, serta ekspansi wilayah dapat dilakukan secara lebih terstruktur.

Selain itu, pembagian permainan ke dalam dua fase memungkinkan strategi ini beradaptasi terhadap dinamika permainan. Fase pertama difokuskan pada eksplorasi dan pembangunan *tower*, sedangkan fase kedua berfokus pada kontrol wilayah dan tekanan terhadap lawan.

4 Implementasi dan Pengujian

4.1 Implementasi 3 Alternatif Solusi

Berikut adalah detail pseudocode untuk ketiga alternatif.

1 Pseudocode Alternatif 1 : Territorial Expansion

```
procedure RUN(rc)                                   ▷ Loop Utama Sistem
    while true do
        turnCount ← turnCount + 1
        switch rc.getType() do
            case SOLDIER: RUNSOLDIER(rc)
            case MOPPER: RUNMOPPER(rc)
            case SPLASHER: RUNSPLASHER(rc)
            default: RUNTOWER(rc) Exception e
        PRINT("Exception Error")
        CLOCK.YIELD
    end while
end procedure
```

```
procedure RUNSOLDIER(rc)
    me ← rc.getLocation()
    if baseLoc = null then INITSECTORS(rc); exploreTarget ← sectorCenter(rc)
    end if
                                                    ▷ Cek Konstruksi Tower
    for tile ∈ senseNearbyMapInfos do
        if tile.hasRuin ∧ canCompletePattern then
            rc.completeTowerPattern(..., tile.loc)
        end if
    end for
                                                    ▷ Movement Logic
    target ← null
    if ∃Enemy then target ← Enemy.loc ∃EnemyPaint target ← EnemyPaint.loc
    ∃EmptyCluster target ← EmptyCluster
    else target ← exploreTarget
    end if
    MOVETOWARD(rc, target)
                                                    ▷ Action Logic
    if canAttack Enemy then ATTACK(Enemy.loc) canAttack EnemyTile AT-
    TACK(EnemyTile) ¬isAlly(me) ATTACK(me)
    end if
end procedure
```

```
procedure RUNMOPPER(rc)
    me ← rc.getLocation()
    closest ← findClosestEnemy(rc, me)
    if closest ≠ null then
```

```
        if dist > 2 then MoveToward(rc, closest.loc)
        end if
        if rc.canMopSwing(dir) then MopSwing(dir); return
        end if
        if rc.canAttack(loc) then Attack(loc); return
        end if
    end if
    ep ← findNearestEnemyPaintTile(rc, me)
    if ep ≠ null then MoveToward(rc, ep)
        if rc.canMopSwing(dir) then MopSwing(dir)
        else Attack(ep)
        end if
    end if
    DoSectorMove(rc)
end procedure
```

```
procedure RunSplasher(rc)
                                                    ▷ Heuristik Serangan Splash
    if rc.getPaint ≥ 50 then
        best ← null, maxScore ← 2 tile ∈ senseInRange(4)
        score ← Hitung skor splash (Enemy=+5, Empty=+1, Ally=-3)
        score ← score + 50
        if mengenai Tower musuh then
            if score > maxScore then best ← tile.loc; maxScore ← score
            end if

            if best ≠ null then Attack(best)
            end if
        end if
                                                                ▷ Movement
        if rc.getPaint < 50 then MoveToward(RefillTarget)
        else MoveToward(EnemyCluster) or DoSectorMove(rc)
        end if

```

2 Pseudocode Alternatif 2

```
function RunUtama(rc)
    // Fase 1: Inisialisasi & Pembaruan Cache Sekutu
    UpdateCacheRobotSekutu(rc)
    homeTower ← CariTowerTerdekat(rc)
    // Fase 2: Pendelegasian Peran
    if not AssignRole(rc) then
        MoveRandom(rc)
        return // Jika gagal mendapat peran, jalan acak
    end if
    // Fase 3: Logistik & Manajemen Cat
    BacaPesanRadar(rc)
```

```
    krisisCat ← TanganiSistemRefill(rc) // Mengurus withdraw, emergency, & needsRefill
    if krisisCat == true then
        return // Prioritas absolut: Hentikan aksi jika butuh cat
    end if
    // Fase 4: Eksekusi Peran Spesifik (State Machine)
    if myRole == SEARCHER then
        RunSearcher(rc)
    else
        if myRole == BUILDER then
            RunBuilder(rc)
        else
            if myRole == SCOUT then
                RunScout(rc)
            else
                if myRole == CONNECTOR then
                    RunConnector(rc)
                else
                    MoveRandom(rc)
                end if
            end if
        end if
    end if
    // Fase 5: Sinkronisasi
    if rc.getRoundNum() (mod 5) == 0 then
        BroadcastPosisiKeSekutu(rc)
    end if
end function
function RunMopper(rc)
    // Fase 1: Manajemen Logistik Cat
    AmbilCatDariTower(rc)
    if ButuhRefill(rc) then
        JalankanRefill(rc)
        return // Hentikan aksi lain jika sedang krisis cat
    end if
    BroadcastAreaBersih(rc)
    // Fase 2: Hierarki Aksi (Greedy Prioritas)
    if CobaTransferCatKeSekutu(rc) then return
    end if
    if CobaSeranganMopSwing(rc) then return
    end if
    targetMop ← CariTargetMopTerbaik(rc)
    if targetMop ≠ null then
        EksekusiAtauKejarMop(rc, targetMop) // Serang jika dekat, kejar jika jauh
        return
```

```
    end if
    // Fase 3: Navigasi Makro
    areaMusuh ← CariAreaMusuhTerbaik(rc)
    if areaMusuh ≠ null then
        BergerakKeAreaMusuh(rc, areaMusuh)
        return
    end if
    // Fallback: Cari sisa noda musuh atau jalan acak
    SapuSisaCatAtauPatroli(rc)
end function
```

```
function RunSplasher(rc)
    // Fase 1: Manajemen Logistik Cat
    TarikCatDariTower(rc) // Reset timer jika berhasil withdraw
    if KondisiKrisisCat(rc) or MasihProsesRefill(rc) then
        JalankanFaseRefill(rc)
        return // Prioritas absolut: Hentikan aksi lain saat butuh cat
    end if
    // Fase 2: Eksekusi Serangan Area (Greedy Splash)
    targetSplash ← CariPusatSplashTerbaik(rc)
    if targetSplash ≠ null then
        if rc.isActionReady() and rc.canAttack(targetSplash) then
            rc.attack(targetSplash)
            BersihkanTargetKosong(targetSplash)
        else
            GerakGreedyMendekat(rc, targetSplash) // Dekati posisi optimal
        end if
        return // Selesai giliran jika ada target serangan
    end if
    // Fase 3: Navigasi Makro (Hierarki Pengejaran)
    klasterMusuh ← CariKlasterCatMusuh(rc)
    if klasterMusuh ≠ null then
        GerakGreedyMendekat(rc, klasterMusuh)
        return
    end if
    musuhTerdekat ← CariRobotMusuhTerdekat(rc)
    if musuhTerdekat ≠ null then
        MoveTo(rc, musuhTerdekat.loc)
        return
    end if
    // Fallback: Kuasai area kosong atau patroli acak
    destinasi ← CariDestinasiKosongTerbaik(rc)
    if destinasi ≠ null then
        GerakGreedyMendekat(rc, destinasi)
    else
```

```
        MoveRandom(rc)
    end if
end function
```

3 Alternatif Solusi 3

```
procedure RunSoldier(rc)
    // Pengecekan Transisi Fase (Late Game)
    if rc.getRoundNum() > 700 then
        skipfirst ← true
    end if
    InitTargetEksplorasi(rc, biasDir, exploreTarget) // Set kompas awal
    ruinKosong ← CariRuinKosong(rc)
    // Prioritas 1: Eksekusi Pembangunan (Greedy)
    if ruinKosong ≠ null and rc.getPaint() > 0 and skipfirst == false then
        // Fungsi abstrak: Berjalan ke Ruin, Mark Pola, Cat Pola, dan Complete
        BangunMenara(rc, ruinKosong.loc)
        return // Selesaikan giliran, fokus penuh pada pembangunan
    end if
    // Prioritas 2: Fase Sebar / Kamikaze
    UpdateTargetEksplorasi(rc, exploreTarget, biasDir)
    // 2a. Eksekusi Gerak (Greedy Navigasi)
    if rc.getMovementCooldownTurns() < 10 then
        GerakHindarRintangan(rc, exploreTarget, biasDir)
    end if
    // 2b. Eksekusi Aksi (Greedy Penguasaan Area)
    if rc.getActionCooldownTurns() < 10 and rc.getPaint() > 0 then
        CatLantaiTerdekat(rc) // Prioritas: Kaki sendiri → Lantai non-sekutu
    end if
end procedure
```

```
procedure RunMopper(rc)
    musuh ← rc.senseNearbyRobots(Musuh)
    // Fase 1: Hierarki Aksi Eksekusi Instan (Greedy)
    if rc.getActionCooldownTurns() < 10 then
        if CekTargetSwing(rc, musuh) ≠ null then
            rc.mopSwingCekTargetSwingrc, musuh // Prioritas 1: Area Damage
        else
            if CekTargetMop(rc, musuh) ≠ null then
                rc.attackCekTargetMoprc, musuh // Prioritas 2: Curi Cat
            else
                if rc.getPaint() > 40 and CekSekutuButuhCat(rc) ≠ null
```

```
then
                rc.transferPaintCekSekutuButuhCat(rc), 20 // Prioritas 3: Support
            end if
        end if
    end if
end if
// Fase 2: Pergerakan (Independen dari Aksi)
if rc.getMovementCooldownTurns() < 10 then
    // Prioritas Arah: Kejar Musuh → Bersihkan Lantai → Acak
    arahGerak ← DapatkanArahPrioritas(rc, musuh)
    GerakHindarRintangan(rc, arahGerak)
end if
// Fase 3: Komunikasi Radar
UpdateEnemyRobots(rc)
end procedure
```

```
procedure RunSplasher(rc)
    myLoc ← rc.getLocation()
    // Fase 1: Serangan Rakus (Prioritas Skor Tertinggi)
    if rc.getActionCooldownTurns() < 10 and rc.getPaint() ≥ 50 then
        bestTarget ← CariTargetSkorMax(rc, myLoc) // Evaluasi skor 1, 2, dan 50
        if bestTarget ≠ null then
            rc.attack(bestTarget)
        end if
    end if
    // Fase 2: Pergerakan (Independen dari Serangan)
    if rc.getMovementCooldownTurns() ≥ 10 then
        return // Sedang cooldown, hentikan fase gerak
    end if
    // 2a. Mode Darurat: Cari Refill
    if rc.getPaint() < 20 then
        return
    end if
    // 2b. Mode Agresif: Kejar Musuh Terdekat
    musuh ← CariMusuhTerdekat(rc)
    if musuh ≠ null then
        GerakJagaJarak(rc, musuh.loc) // Jarak optimal untuk splash
        return
    end if
    // 2c. Mode Sapu Bersih: Cari Lantai Bukan Sekutu
    targetLantai ← FindNearestNonAllyTile(rc)
    if targetLantai ≠ null then
        GerakKeTarget(rc, targetLantai)
        return
```

```
    end if
    // 2d. Fallback: Eksplorasi Kompas (Jalan lurus, belok jika mentok)
    myExploreDir ← JalanDenganKompas(rc, myExploreDir)
end procedure
```

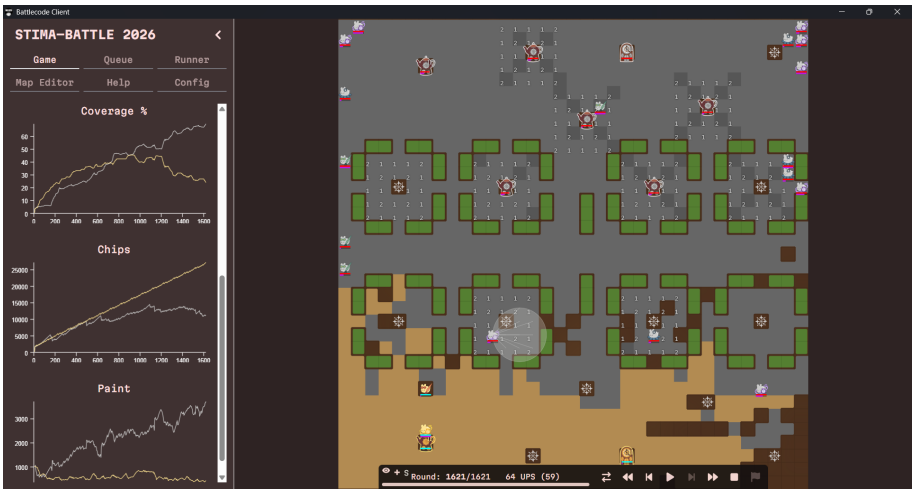
4.2 Struktur Data, Fungsi, dan Prosedur

3 Struktur Data dan Fungsi Prioritas Algoritma Bot Battlecode 2025

File	**Komponen**	**Kegunaan dan Deskripsi Singkat**
Mopper	enemyAreaTargets	Daftar lokasi cat musuh dari Searcher.
Mopper	checkAndBroadcastCleared	Mengirim sinyal MSG_CLEARED_AREA jika area sudah bersih.
Mopper	findBestMopTarget	Heuristik target serangan.
Mopper	tryOptimalMopSwing	Serangan area jika mengenai lebih dari satu robot musuh.
Soldier	myRole	Penentu peran bot: Searcher, Builder, Scout, atau Connector.
Soldier	history	Jejak langkah untuk mencegah gerakan berulang.
Soldier	scanAndReport	Searcher mendeteksi ruin dan kluster kosong bagi tim.
Soldier	tryBuildPaintTower	Algoritma Builder mewarnai pola 5x5 dan membangun Tower.
Soldier	findConnectorTarget	Mencari tile terbaik untuk menyambung jalur cat.
Splasher	ScoredLocation	Penyimpan koordinat target beserta skor efektivitas ledakan.
Splasher	findBestSplashCenter	Mencari titik tengah ledakan dengan prioritas Tower musuh (+50) dan cat musuh (+5).
Splasher	findBestEnemyPaintCluster	Mencari area cat musuh terpadat.
Splasher	run	Navigasi agresif mengejar unit atau kluster cat lawan.
Splasher	readMessages	Mendengarkan koordinat kluster kosong untuk arah ekspansi.
Tower	knownRuins	Daftar ruin belum dibangun untuk dibagikan ke Builder.
Tower	dedupData	Cache untuk mencegah infinite echo antar Tower di radio.
Tower	relaySymmetricRuins	Memancarkan 3 lokasi simetris ruin berdasarkan asumsi peta.
Tower	spawnPhase2	*Spawn* unit bergantian dan pengerahan Builder darurat.
Tower	broadcastSelf	Mengumumkan lokasi Tower sebagai titik isi ulang cat.
Navigation	knownPaintTowers	Array lokasi Paint Tower kawan yang pernah dibuat.
Navigation	blacklistedTowers	Daftar Tower dihindari karena cat habis atau tidak responsif.
Navigation	moveTo	Gerak utama dengan mode Tracing (Bug Navigation) jika terhalang.
Navigation	findBestRefillTower	Mencari lokasi pengisian cat terbaik.
Navigation	tryWithdrawFromTower	Penarikan cat saat unit bersebelahan dengan Tower.

4.3 Pengujian (Bot Utama vs Bot Alternatif)

Bot Utama vs Bot Alternatif 1 (Territorial Expansion)



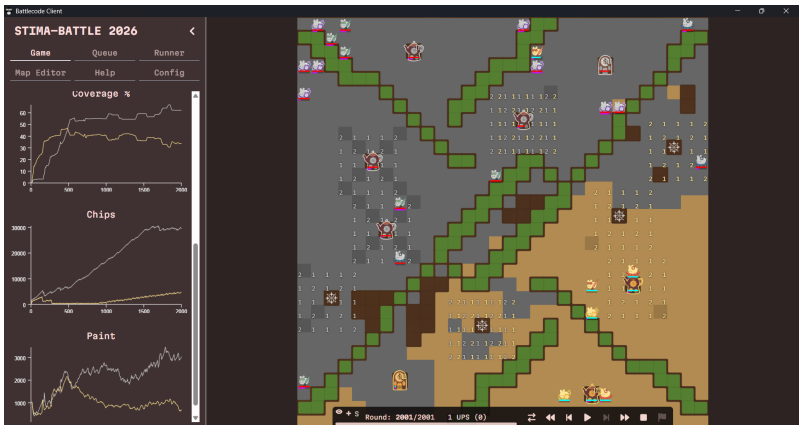
3 Match 1: Team A (Bot Utama) Menang

3 Match 2: Team A (Bot Utama) Menang

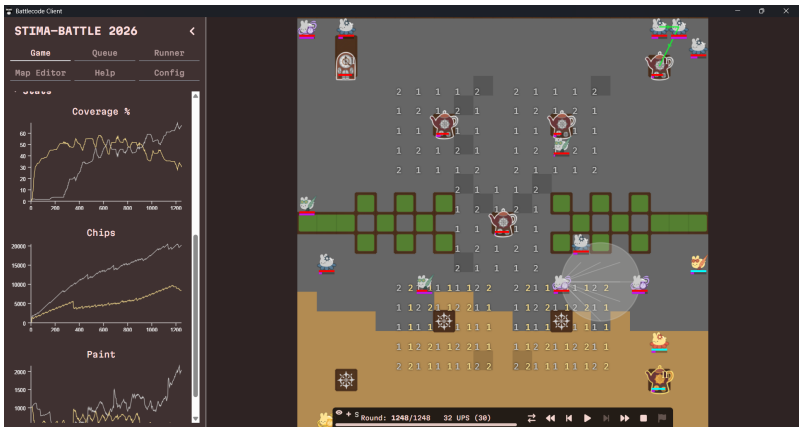
3 Match 3: Team B (Bot Alternatif 1) Menang

Pengujian	Team A (Bot Utama)	Team B (Bot Alternatif 1)	Pemenang
Match 1	Menang	Kalah	Team A
Match 2	Menang	Kalah	Team A
Match 3	Kalah	Menang	Team B

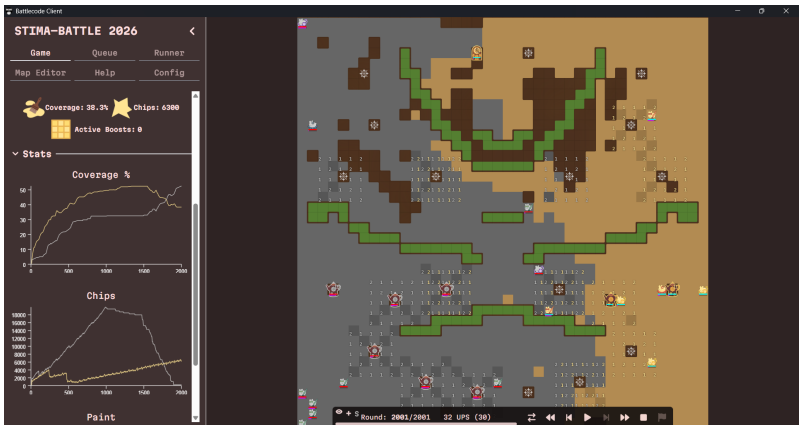
Bot Utama vs Bot Alternatif 2 ((Greedy Prioritas Aksi Eksplorasi)



3 Match 1: Team A (Bot Utama) Menang



3 Match 2: Team A (Bot Utama) Menang



3 Match 3: Team A (Bot Utama) Menang

Pengujian	Team A (Bot Utama)	Team B (Bot Alternatif 2)	Pemenang
Match 1	Menang	Kalah	Team A
Match 2	Menang	Kalah	Team A
Match 3	Menang	Kalah	Team B

4.4 Analisis Hasil Pengujian

Berdasarkan hasil pengujian yang telah dilakukan, algoritma *Greedy* pada Bot Utama menunjukkan performa yang dominan dengan beberapa karakteristik kunci yang menarik untuk dibahas:

- Mayoritas kemenangan yang diraih oleh Bot Utama merupakan skema *comeback*. Strategi ini berfokus pada pembangunan *tower* yang masif di awal permainan. Meskipun pada mulanya Bot Utama tidak menguasai petak sebanyak lawannya, pendekatan visioner ini memungkinkan Bot Utama untuk memetik hasilnya dan melakukan *overtake* skor di akhir pertandingan (*late game*).
- Terdeteksi satu kelemahan signifikan pada Bot Utama saat menghadapi lawan dengan karakteristik seperti Bot Alternatif 1, terutama pada ukuran *map* yang

kecil. Dalam kondisi *map* yang sempit, *coverage area* lawan dapat meningkat jauh lebih cepat sehingga Bot Alternatif 1 mampu mengamankan kemenangan sebelum Bot Utama mencapai potensi maksimalnya, sebagaimana terlihat pada hasil pertandingan ketiga.

- Efektivitas algoritma *Greedy* Bot Utama meningkat secara signifikan apabila posisi *tower* pada *map* dapat diakses dengan mudah atau memiliki pola yang teratur. Kondisi tersebut mempermudah Bot Utama dalam melakukan monopoli *tower* sejak awal laga, yang secara langsung memperkuat kontrol wilayah dan dominasi permainan.

5 Kesimpulan dan Saran

5.1 Kesimpulan

Secara keseluruhan, algoritma yang dipilih cukup berhasil dalam memenangkan permainan dibandingkan algoritma alternatif lainnya. Algoritma *greedy* yang diimplementasikan pada algoritma ini sebagian besar berfungsi dengan baik. Namun, selain itu fitur-fitur lain pada permainan juga membantu pengambilan keputusan secara *greedy*. Algoritma yang dipilih sangat memaksimalkan penyampaian informasi dari bot-bot pembantu (*Searcher*, *Scout*) untuk memilih pengambilan keputusan lokal untuk mencapai global.

5.2 Saran

Untuk pengembangan selanjutnya, algoritma yang dipilih ini dapat dikembangkan menjadi sebagai berikut.

1. Bangun *Defense Tower* di garis depan *map* agar dapat membantu pengambilalihan wilayah lawan.
2. Optimasi kembali fungsi *greedy* yang digunakan dalam pengecatan wilayah.
3. Eksplorasi komposisi *spawning* bot lainnya.
4. *Handle* beberapa *bug*, seperti bot menabrak dinding.

Lampiran

- **Repository GitHub:** `https://github.com/Akram17t/Tubes1_IndonesiaSuksesLihatAkramManu`

Pustaka

[1] R. Munir, "Algoritma Greedy (Bagian 1)," Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung, 2026. [Online]. Available: `https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/04-Algoritma-Greedy-(2026)-Bag1.pdf`. [Accessed: Mar. 4, 2026].

[2] MIT Battlecode, "Battlecode Official Website," [Online]. Available: `https://battlecode.org`. [Accessed: Mar. 9, 2026].

[3] MIT Battlecode, "Battlecode 2025 Java Scaffold and Quick Start," [Online]. Available: `https://play.battlecode.org/bc25java/quick_start`. [Accessed: Mar. 11, 2026].